18-786 HW5: Generating Cat Images with GANs

Yizhen Xu
Andrew ID: yizhenxu
18-786 Spring 2026

April 14, 2026

1 Part 1: Deep Convolutional GAN (DCGAN)

A DCGAN is implemented to generate 64×64 Grumpy-Cat images from the `grumpifyBprocessed` dataset (204 images). The generator is built with `Upsample`+`Conv2d` blocks (the first block uses a direct 4×4 convolution to turn the 1×1 noise into a 4×4 feature map), and the discriminator is a stack of stride-2 convolutions that compresses the image down to a single logit. LSGAN-style mean-square loss is used in the training loop.

1.1 Training Hyperparameters

Hyperparameter	Value
Image size	64×64
Noise size $\|z\|$	100
Base conv width (`conv_dim`)	32
Batch size	16
Optimizer	Adam ($\beta_1 = 0.5,\ \beta_2 = 0.999$)
Learning rate	2×10^{-4}
Loss	LSGAN (mean-squared)
Normalization	InstanceNorm
Epochs	500 $\Rightarrow$ ≈6,500 iterations
Sample / checkpoint cadence	every 200 / 400 iter
Random seed	11

Table 1: DCGAN training hyperparameters.

1.2 Data Augmentation

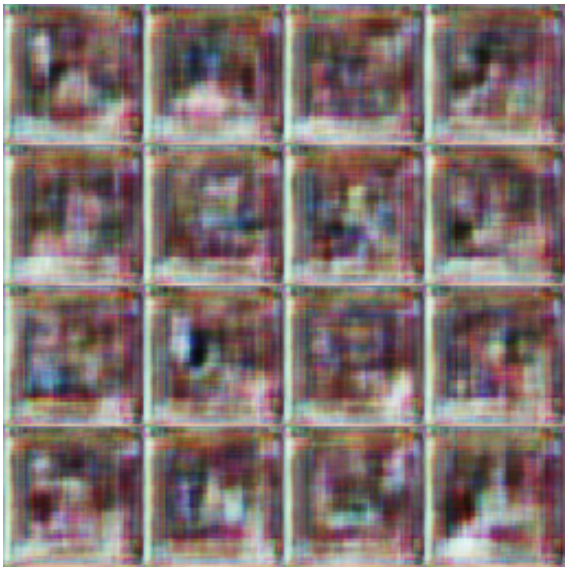
Two preprocessing pipelines are compared:

- **Basic:** `Resize(64)` $\rightarrow$ `ToTensor` $\rightarrow$ `Normalize`$(0.5, 0.5)$.
- **Advanced:** `Resize(70)` $\rightarrow$ `RandomCrop(64)` $\rightarrow$ `RandomHorizontalFlip` $\rightarrow$ `ColorJitter`$(0.1, 0.1, 0.1)$ $\rightarrow$ `ToTensor` $\rightarrow$ `Normalize`$(0.5, 0.5)$.

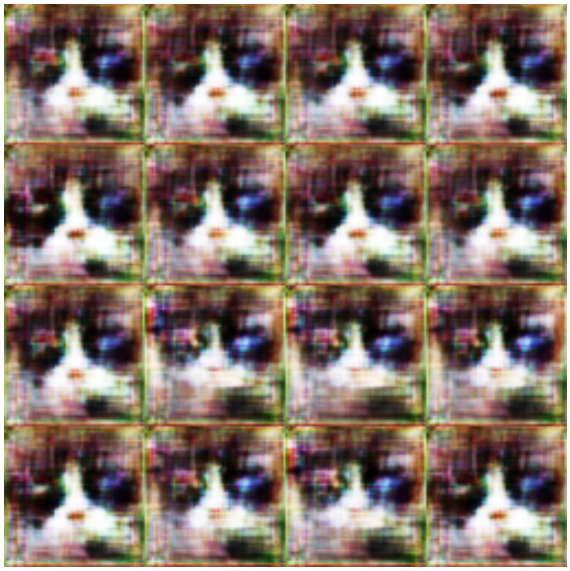
The advanced pipeline introduces random spatial and photometric perturbations, which make it much harder for the discriminator to memorize the small training set.

1.3 Generated Samples

Figure 1 and Figure 2 show 4×4 sample grids at the beginning (iter 400) and end (iter 6000) of training for both pipelines. All grids are generated from the same *fixed* noise batch so that differences reflect the state of the generator rather than noise variability.



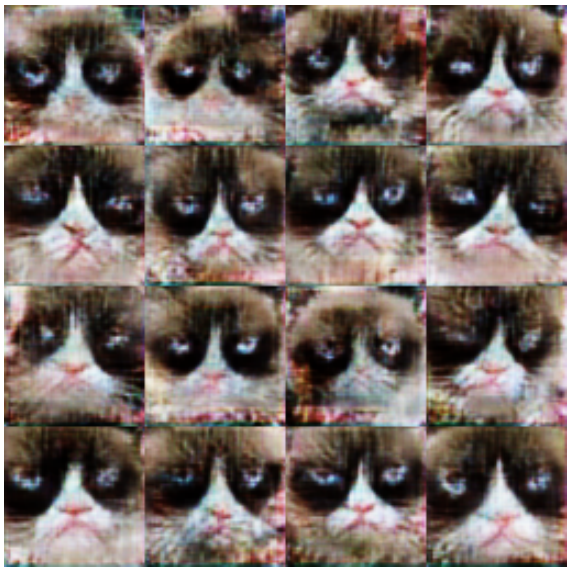
(a) Basic augmentation, iter 400.



(b) Basic augmentation, iter 6000.

Figure 1: Generator samples with **basic** augmentation. At iter 400 the output is mostly noise; at iter 6000 the generator has collapsed to nearly identical black-and-white cat faces across all 16 grid cells (mode collapse).

(a) Advanced augmentation, iter 400.



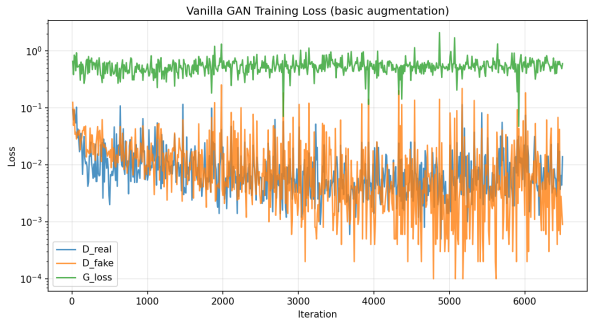
(b) Advanced augmentation, iter 6000.

Figure 2: Generator samples with **advanced** augmentation. At iter 400 the rough positions of the eyes, nose and mouth are already visible. At iter 6000 the generator produces clearly recognizable Grumpy-Cat faces with distinct ears, eyes and a frowning mouth, and the 16 cells show visibly more variation than the basic run (less mode collapse).

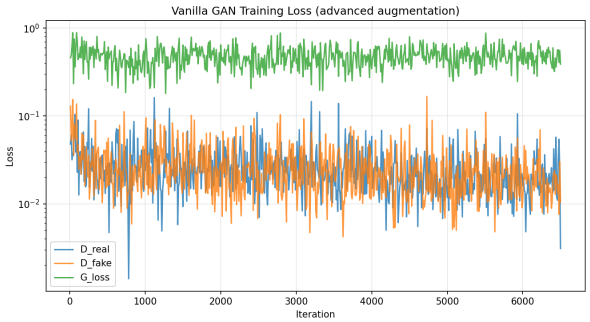
Visual comparison. Advanced augmentation produces images with (i) *less noise* (edges are sharper, background artifacts are reduced), (ii) *less mode collapse* (grid cells differ in pose and color), and (iii) *more cat features* (Grumpy-Cat-specific fur pattern, ears, eyes and nose are clearly rendered). This matches the "visually better" criterion specified in the assignment.

1.4 Training Loss Curves

Figure 3 plots D_{real}, D_{fake} and G_{loss} (log scale on the y-axis) for the two runs.



(a) Basic augmentation.



(b) Advanced augmentation.

Figure 3: Training loss curves (log-y) for the Vanilla-style DCGAN (plot script labels it "Vanilla GAN"; the architecture is the DCGAN described in Section 1). The high-frequency oscillation is expected — it reflects the adversarial tug-of-war between G and D.

1.5 Analysis

Oscillation is a healthy signature. Unlike supervised training, GANs minimize a min–max objective; every step that strengthens D forces G to adapt, and vice versa. The losses therefore oscillate at high frequency throughout training. Neither run shows divergence (no loss explodes) nor collapse (no loss flatlines at zero), so the adversarial dynamics themselves are well-behaved.

The key signal: D_{fake} behaves differently in the two runs. Table 2 summarizes the order of magnitude of D_{fake} through training:

	iter 1000	iter 2000	iter 4000	iter 6000
D_{fake}, basic	0.021	0.003	0.0013	0.0013
D_{fake}, advanced	0.058	0.030	0.029	0.011

Table 2: D_{fake} at representative iterations (values taken from `results/log_basic.txt` and `results/log_advanced.txt`).

In the basic run, D_{fake} drops by nearly two orders of magnitude; the discriminator has effectively memorized the 204 real images and can reject every fake sample. Once D is nearly perfect, the gradient that flows back to G vanishes and the generator settles on a single low-variance mode — exactly the phenomenon observed visually at iter 6000.

In the advanced run, the random crops, flips and color jitter force D to see a different version of each real image every iteration. D can no longer memorize and is pushed to learn more generalizable features. D_{fake} consequently stays in the 10^{-2} range, keeping a non-trivial gradient flowing into G. The generator keeps improving, and mode collapse is avoided.

G_{loss} is similar in both runs, and that is expected. Under the LSGAN objective $G_{\text{loss}} = \frac{1}{2}\mathbb{E}[(D(G(z)) - 1)^2]$ is bounded in $[0, 0.5]$ as long as D's outputs stay in $[0, 1]$. Its absolute value is a poor indicator of generator quality; what matters is D_{fake} together with the visual samples. Both runs sit around $G_{\text{loss}} \approx 0.4$–$0.6$, but describe very different regimes: in basic, G has collapsed to a point that D can trivially reject; in advanced, G genuinely tracks the real distribution.

Practical heuristics confirmed.

- $D_{\text{real}} \approx D_{\text{fake}} \approx 10^{-2}$ with $G_{\text{loss}} \in [0.3, 0.7]$ is the healthy equilibrium observed in the advanced run.
- A joint decay of both D_{real} and D_{fake} to $\leq 10^{-3}$ is an early warning of D overfitting and imminent mode collapse, as seen in the basic run.

1.6 Conclusion for Part 1

The DCGAN implementation successfully generates recognizable Grumpy-Cat images. On the tiny (204-image) training set, basic preprocessing lets the discriminator overfit, leading to mode collapse after a few thousand iterations. Advanced data augmentation restores the G–D balance: the generator produces sharper, more diverse and more cat-like samples at iter 6000, while the training losses remain in a healthy oscillating regime.

2 Part 2: Architectures and Objective Functions

Part 2 compares four GAN variants implemented from scratch (no use of `torch.nn.utils.spectral_norm` or similar wrappers). All four variants share the same G backbone used in Part 1 (except the custom architecture) and are trained for ≈6,500 iterations using the advanced data augmentation pipeline. The four variants are:

1. **LSGAN** (Mao *et al.*, 2017) — MSE-based objective; identical architecture to the DCGAN baseline, only the loss is changed.

2. **Spectral Normalization (SN)** (Miyato *et al.*, 2018) — power-iteration-based spectral normalization applied to every convolution in the discriminator, turning D into a 1-Lipschitz network.

3. **WGAN-GP** (Arjovsky 2017, Gulrajani 2017) — the discriminator becomes a critic with a real-valued score; the loss is the Wasserstein-1 dual plus a gradient penalty $\lambda\,\mathbb{E}[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2]$ on random interpolations $\hat{x} = \varepsilon x + (1-\varepsilon)G(z)$. The critic is updated n_{critic}=5 times per generator step; Adam uses β_1=0.0, β_2=0.9.

4. **Custom** — a residual, spectrally-normalized architecture: `ResnetBlock`s are inserted after each upsample in G, and the discriminator uses SN on every convolution plus two SN `ResBlock`s at intermediate resolutions. Trained with the LSGAN objective.

2.1 Training Hyperparameters (Part 2)

	LSGAN	SN	WGAN-GP	Custom
Loss	MSE	MSE	Wasserstein + GP	MSE
Adam (β_1, β_2)	$(0.5, 0.999)$	$(0.5, 0.999)$	$(0.0, 0.9)$	$(0.5, 0.999)$
n_{critic}	1	1	5	1
λ_{GP}	—	—	10	—
Norm in D	InstanceNorm	none (SN only)	LayerNorm (GN_1)[1]	none (SN only)

Table 3: Part 2 variant-specific hyperparameters. Image size, noise size, batch size, learning rate and total iterations are the same as Part 1.

2.2 Generated Samples at Iteration 6000

Figure 4 shows the 4×4 sample grids produced by each variant at the end of training, all generated from the same fixed noise batch.

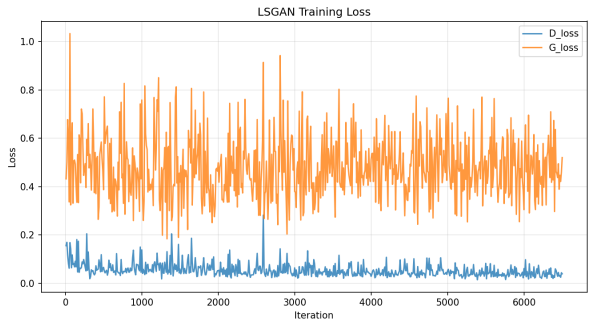
[1] GN_1 denotes `GroupNorm(num_groups=1, C)`, which is mathematically equivalent to per-sample `LayerNorm` over the spatial dimensions. This is the normalization recommended by the WGAN-GP paper, since BatchNorm breaks the per-sample gradient penalty.

(a) LSGAN. (b) Spectral Norm. (c) WGAN-GP. (d) Custom.

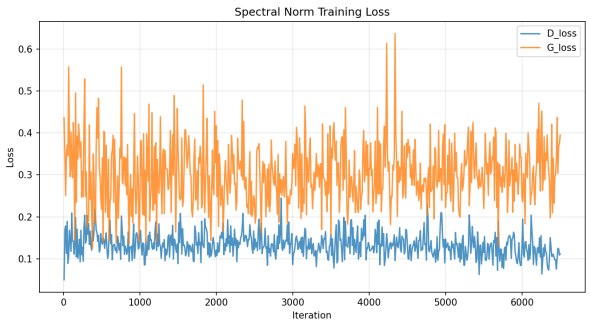
Figure 4: Generator samples at iter 6000 for the four Part 2 variants (fixed noise). LSGAN and SN produce recognizable faces with mild background artifacts; WGAN-GP shows stronger texture noise; the Custom architecture yields the sharpest, cleanest results with clearly rendered white chest fur, chin, frowning mouth and ear shape.

2.3 Training Loss Curves

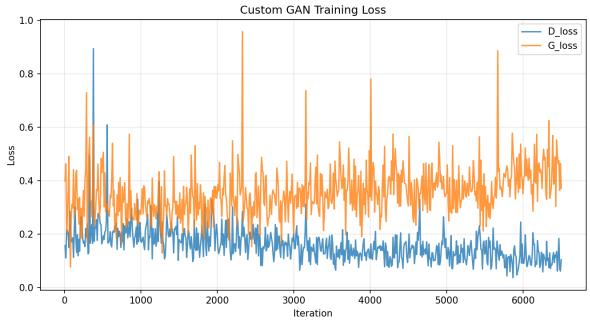
Figures 5–6 show the training losses for each variant. The LSGAN-style variants (LSGAN, SN, Custom) share the same loss axes; WGAN-GP is plotted separately because its loss is an unbounded real-valued Wasserstein estimate and additionally provides two diagnostic quantities (the estimated Wasserstein distance and the gradient penalty).



(a) LSGAN.



(b) Spectral Norm.



(c) Custom.

Figure 5: LSGAN-style variants: D_{loss} and G_{loss} throughout training. Enlarged for readability; WGAN-GP is plotted separately in Figure 6 because its loss sits on a different (signed, unbounded) scale.

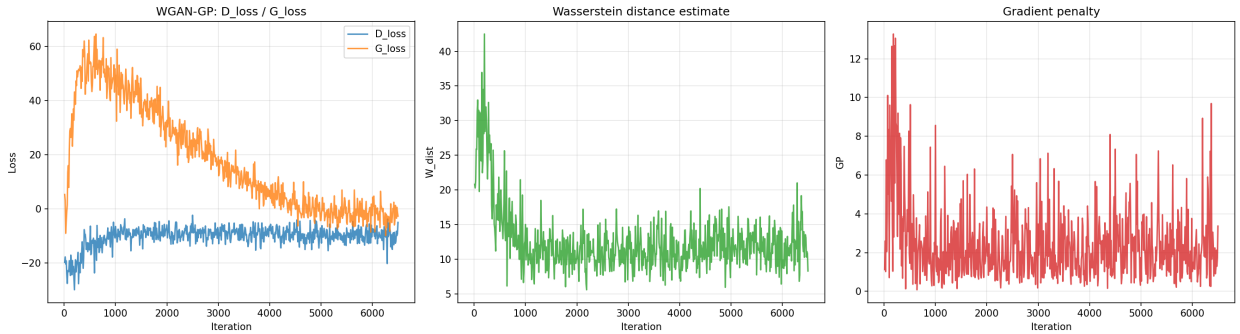


Figure 6: WGAN-GP diagnostics: (left) D_{loss} and G_{loss}, (middle) the Wasserstein distance estimate $\hat{W} = \mathbb{E}[D(\text{real})] - \mathbb{E}[D(\text{fake})]$, (right) the gradient penalty term.

2.4 Analysis

D_{loss} magnitudes reveal which variant best balances G and D. Averaging over the second half of training, the MSE-based variants settle at

- LSGAN: $D_{\text{loss}} \approx 0.04$, $G_{\text{loss}} \approx 0.45$.
- Spectral Norm: $D_{\text{loss}} \approx 0.10$, $G_{\text{loss}} \approx 0.38$.
- Custom: $D_{\text{loss}} \approx 0.10$, $G_{\text{loss}} \approx 0.40$.

The discriminator loss under SN and Custom is *2–3× larger* than under plain LSGAN. This is exactly the effect predicted by the SN paper: the 1-Lipschitz constraint prevents D from becoming arbitrarily confident, which keeps a non-trivial gradient flowing into G. Consistent with this, the generator loss is slightly *lower* for SN/Custom: because the Lipschitz-bounded discriminator has limited expressive power (output values cannot grow arbitrarily far from each other across inputs), the same generator can reach a higher $D(G(z))$ score — not because G is qualitatively better, but because D is constrained to be smoother.

Stability ranking. Because WGAN-GP uses a signed, unbounded Wasserstein loss whereas the other three use bounded MSE losses in $[0, 1]$, it is not meaningful to compare their oscillation amplitudes directly. The ranking is therefore given in two groups.

Among the MSE-based variants (smoothest first):

1. **Spectral Norm** — cleanest, smallest-amplitude oscillation.
2. **LSGAN** — healthy but noisier peaks up to $G \approx 1.0$.
3. **Custom** — SN-level baseline with occasional spikes to ≈ 0.9; these are caused by the additional ResBlock depth but decay quickly and never cause divergence.

WGAN-GP is evaluated on its own terms (see next paragraph): its raw D_{loss} spans tens of units and cannot be put on the same axes, but the Wasserstein-distance estimate $\hat{W}$ and the gradient-penalty term give direct diagnostics of stability.

WGAN-GP: what the three diagnostic panels mean.

- The *left* panel shows G_{loss} (orange) descending from roughly +60 toward 0 during the first 4,000 iterations. Unlike the MSE variants where G_{loss} only oscillates, the WGAN generator loss is directly proportional to $-\mathbb{E}[D(G(z))]$; its monotone decrease is a meaningful signal that the critic increasingly accepts the generator's samples.
- The *middle* panel plots the Wasserstein distance estimate. Ideally this should decrease monotonically toward a small constant. It decreases from $\approx$40 to the range $[8, 13]$ but is still oscillating at iter 6,500 — the model is *not fully converged* yet. This is consistent with the small training set (204 images) and the fact that, with n_{critic}=5, the generator has only performed $\approx$1,300 update steps.
- The *right* panel plots the gradient penalty. It falls from $\approx$12 to the range $[1, 5]$, showing that the 1-Lipschitz constraint is approximately, but not perfectly, satisfied.

The visual samples are therefore coarser than those of SN / Custom, and this under-training is the likely cause — not a flaw in the WGAN-GP method itself.

Custom wins visually despite noisier losses. The Custom architecture exhibits more loss spikes than SN, yet produces the sharpest samples of all four variants. This is a useful reminder that loss-curve smoothness and sample quality are *not* the same thing: the deeper residual G makes optimization bumpier in exchange for a richer function class. In this experiment the trade-off is favorable.

2.5 Conclusion for Part 2

All four variants produce recognizable Grumpy-Cat images with the advanced augmentation pipeline. Spectral normalization demonstrably stabilizes training (smoother losses, higher equilibrium D_{loss}) without harming quality. The Custom architecture — combining spectral normalization with residual blocks in both G and D — yields the best visual quality, confirming that architectural improvements compound with stabilization techniques. WGAN-GP produces meaningful but visibly undertrained samples within the 6,500 iterations budget; its descending G_{loss} and shrinking gradient penalty suggest that longer training would close the gap.

Final visual-quality ranking. Considering sharpness, absence of background artifacts and the correct rendering of cat-specific features (chest fur, ears, frowning mouth), the end-of-training samples rank as

$$\text{Custom} \;>\; \text{SN} \;\approx\; \text{LSGAN} \;>\; \text{WGAN-GP},$$

with the caveat that WGAN-GP's last-place finish reflects under-training rather than an intrinsic weakness of the objective.

References

[1] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. In *NeurIPS*, 2014.

[2] A. Radford, L. Metz, and S. Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. In *ICLR*, 2016.

[3] X. Mao, Q. Li, H. Xie, R. Y. K. Lau, Z. Wang, and S. P. Smolley. Least squares generative adversarial networks. In *ICCV*, 2017.

[4] M. Arjovsky, S. Chintala, and L. Bottou. Wasserstein generative adversarial networks. In *ICML*, 2017.

[5] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville. Improved training of Wasserstein GANs. In *NeurIPS*, 2017.

[6] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida. Spectral normalization for generative adversarial networks. In *ICLR*, 2018.